C Standard Library

CSC 230 : C and Software Tools

NC State Department of Computer Science

Topics to Discuss

- The C Standard Library
  - More I/O functions
  - Error handling
  - More String Functions
  - Random Numbers
- Environment Variables
- Variable numbers of parameters
- Longjmp

The C Standard Library

- A small set of useful functions, standardized across all platforms
- Function declarations, macros and type definitions spread across **24 Header Files**
- You've already seen several of these:

Header	Description
limits.h	Constants for implementation-specific properties of integer types
stdbool.h	Short names for the bool type.
string.h	Manipulating strings
ctype.h	Classify character types, convert uppercase ← → lowercase
assert.h	Error checking mechanism for commenting and debugging

The C Standard Library

- You'll meet some new headers
- And learn some more about some old friends.

stdio.h	Text and binary input/output to the terminal and files
stdlib.h	Various utilities: memory allocation, random numbers, sorting
errno.h	Error codes reported by library functions and the OS
math.h	Common mathematical constants and functions
stdarg.h	Working with variable numbers of arguments, like printf() does.
setjump.h	Support for non-local exit from a function (kind of like an exception)

The C Standard Library

- There are lots more that we won't get to meet.

locale.h	Choosing behaviors based on where you are
time.h	Time & date calculations and format conversions
wchar.h	Wide character support, for non-Latin character sets
wctype.h	Like ctype.h, but for wide characters.
signal.h	Asynchronous interaction between C program and the OS
stddef.h	Useful macros and types
complex.h	Functions for working with complex numbers
tgmath.h	Type-generic macros for working with either floating-point or complex numbers

The C Standard Library

fenv.h	Control over the floating-point computation environment
float.h	Constants for implementation-specific properties of floating point numbers and calculations
inttypes.h	Precise conversion between integer types
iso646.h	For programming with an ISO 646 variant character set, where some standard C symbols may not be available.
stdint.h	Integer types with standard bit widths

More I/O Functions

- We've learned the most important I/O functions
  - There are more we haven't talked about yet.
- Read a line of text from stdin

```
char *gets( char str[] );
```

I'm a bad function.
Don't ever use me.

- Read a line from any stream.

```
char fgets( char str[], int size, FILE *fp );
```

- Write a string to standard output.

```
int puts( char str[] );
```

- Write a string to any stream.

```
int fputs( char str[], FILE *fp );
```

More I/O Functions

- Functions for managing your files.
- Remove a file:

```
int remove( const char pathname[] );
```

- Rename an existing file:

```
int rename( const char old[], const char new[] );
```

- Make a temporary file that goes away when closed:

```
FILE *tmpfile( void );
```

Checking for EOF

- We have a simple, general technique for detecting a successful read.

```
while ( scanf( "%d", &v ) == 1 ) …
```

  - Call a reading function … see if it returns successfully.
  - Remember when you fail or reach EOF.
- LibC remembers this for us automatically.

```
int feof(FILE *stream);
```

  - Returns true if you've already reached the EOF.
  - Set whenever a reading function encounters EOF (even internally).

Easy Mistakes With EOF

- This gives us a new, terrible way to try to detect the end of input.

```
while ( ! feof( input ) ) {
  fscanf( input, "%d", &value );
  printf( "got: %d\n", value );
}
```

This is bad.
Don't do this.

```
25 102EOF
```

EOF right after the second number.

output

```
25
102
```

```
25 102\n
EOF
```

Some whitespace before the EOF.

output

```
25
102
102
```

Things go Wrong

- A file can encounter an error condition.
  - If you try to write to a file that's open for reading.
  - Or the other way around
  - Or the disk is full
  - Or you reach the maximum files size.
- When this happens, the stream notices and remembers it.
  - If you know how to ask, it will tell you something already went wrong:

```
int ferror( FILE *stream );
```

Forgetting Past Mistakes

- The EOF and error conditions remain set
- … until you close the file
- … or explicitly clear them.
- There's a function for that:

```
void clearerr(FILE *stream);
```

Things go Wrong

- C has a simple technique for handling errors
- And a header to go along with it:

```
#include <errno.h>
```

Secretly, this is no longer really a global variable. That would be bad. But, for now, pretend it is.

  - It exposes a global variable errno
  - An integer that gets set to indicate numerous error conditions

Try something that might fail.

```
if ( someLibraryFunction() != SUCCESS ) {
   if ( errno == EACCES ) {
     fprintf( stderr, "You can't touch this\n" );
     …
```

Check what went wrong

Documentation

Tells You What to Expect

```
$ man fopen
FOPEN(3)                    Linux Programmer's Manual                    FOPEN(3)

NAME

       fopen, fdopen, freopen - stream open functions

SYNOPSIS

       #include <stdio.h>

       FILE *fopen(const char *path, const char *mode);

…

ERRORS
       EINVAL The mode provided to fopen(), fdopen(), or freopen() was
              invalid.

       The fopen(), fdopen() and freopen() functions may also fail and set
       errno for any of the errors specified for the routine malloc(3).

…
```

Making Sense of errno

- There are numerous values errno may be given (EACCESS, EEXIST, EFAULT, ENOENT, ENOMEM, ENOSPC …)
- There's a function that knows short messages for each of these:

```
void perror( const char *s );
```

Message of your choice, included with the error report.

Using perror()

- We use perror() like this:

```
FILE *fp = fopen( “someFile.txt”, “r” );
if ( fp == NULL ) {
  perror( “someFile.txt” );
  exit( 1 );
}
```

- And we get:

```
$ myProgram
someFile.txt: No such file or directory.
```

More String Functions

- **`string.h`** declares lots of useful functions for working with strings and memory.
  - An easy way to copy memory between arrays:

```
void *memcpy( void *dest, void *src, size_t n );
```

  - An easy way to move memory within an array:

```
void *memmove( void *dest, void *src, size_t n );
```

  - An easy way to fill memory with a given byte value:

```
void *memset( void *, int c, size_t n );
```

Using memset()

- **`memset()`** can give you an easy way to zero-fill an array.

```
int *list = (int *) malloc( n * sizeof( int ) );
memset( list, 0, n * sizeof( int ) );
```

We could have done this with calloc().

- Works for any memory region.

```
int list[ n ];
memset( list, 0, n * sizeof( int ) );
```

Initialization syntax doesn't work with a variable-sized array.

Using memset()

- Beware, you can only fill memory with byte values.

This will fill **bytes** with 1, not an int 1.

```
int *list = (int *) malloc( n * sizeof( int ) );
memset( list, 1, n * sizeof( int ) );
```

- Filling with a given char works fine:

```
char str[ n + 1 ];
memset( str, '.', n );
str[ n ] = '\0';
```

Fill with n dots.

Still need a null terminator.

More String Functions

- Lots of string functons return a char pointer. An easy way to copy memory between arrays:

```
char *strcpy( char *dest, char *src );
char *strcat( char *dest, char *src );
```

- What's this for?
  - Generally, it's to let you chain together multiple string operations.

```
strcat( strcpy( dest, "hello" ), " world" );
```

Maybe a little inefficient.

String Searching Functions

- You can look for a specific character in a string:

```
char *strchr( char const *str, char ch );
```

Returns a pointer to the first occurrence, or NULL if unsuccessful.

Even works for finding the null terminator.

- You could use this to find (and replace) a particular character.

```
if ( strchr( str, " " ) ) …
```

Is there a space?

```
if ( ( p = strchr( str, "B" ) ) )
  *p = 'b';
```

Replace ‘B’ with ‘b’

String Searching Function

- You can look for one of a set of characters.

```
char *strpbrk( char const *str, char const *accept );
```

Pointer to first match, or NULL.

Look for any character in this set.

- You could find lots of ways to use this:

Look for any of these punctuation characters.

```
if ( strpbrk( str, ".,;:?!" ) ) …
```

```
while ( ( p = strpbrk( p, "0123456789" ) ) )
    *p = 'X';
```

Replace digits with X.

String Searching Function

- You can look for a whole string:

```
char *strstr( char const *haystack,
              char const *needle );
```

Easy to remember what these are for.

Pointer to first match, or NULL.

- You could find lots of ways to use this:

```
while ( ( p = strstr( p, "CSC 230" ) ) )
  memcpy( p, "C Class", 7 );
```

Could I have used strcpy() here?

Replace occurrences of "CSC 230" with "C Class"

Random Numbers

- **`stdlib.h`** provides functions and constants for pseudo-random number generation:
  - Generate a random integer 0 … RAND_MAX

```
int rand( );
```

  - Constant for the maximum random value

```
RAND_MAX
```

  - Provide a seed for the pseudo-random sequence

```
void srand( unsigned int seed );
```

Choosing a Seed

- By default, you will get the same pseudo-random sequence at every execution.
  - Unlike Math.random() in Java.
- A common remedy, seed the sequence with the current time:

```
void srand( time( NULL ) );
```

I'm declared in time.h

Number of seconds since the start of the universe, January 1, 1970.

Example

- To generate a number in [0.0,1.0]

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

double getrand() {
    int r = rand();
    return (double) r / RAND_MAX;
}

int main() {
    srand( time(NULL) );
   …
    double r = getrand();
   …
```

Random Numbers

- We could generalize this technique to get any range we need.
- Say we want a real number between **`min`** and **`max`** (inclusive)

```
double min = …, max = … ;
double range = max - min;

double r1 = (double) rand() / RAND_MAX * range;
double r2 = r1 + min;
```

Using Environment Variables

- Your execution environment supports variables
  - We can access these from the shell

```
$ echo $HOME
/home/jerry
$ NAME=linda
$ echo $NAME
linda
```

Shell syntax to get the value of an environment variable.

Or change the value of a variable.

- The **env** command will show you all your environment variables.

```
$ env
HOSTNAME=engr-ras-206.eos.ncsu.edu
SHELL=/bin/bash
EDITOR=vi
...
```

Environment Variables

- There are lots of environment variables with standard values:

Variable	Definition
HOME	Path to your home directory
USER	Current username
PATH	List of directories to check for programs you try to run.

- The environment isn't just for the shell.
- It's a name to value mapping available to all C programs.

Using the Environment

- The standard library lets us get/set these variables
  - We need our good friend, `stdlib.h`
  - Then, we can get environment variable values

```
char *str = getenv( "HOME" );
printf( "HOME = %s\n", str );
```

  - And, we can change them

```
setenv( "HOME", "/some/other/place", 1 );
```

Variable to set.

new value.

Should I overwrite?

- These changes are local to the program, they go away when it exits.

Using the Environment

- You can use standard environment variables in your programs.

```
int main( int argc, char *argv[] )
{
  printf( "Hello %s\n", getenv( "USER" ) );
  return EXIT_SUCCESS;
}
```

```
$ ./program
Hello dbsturgi
```

Using the Environment

- You can use your own environment variables.

```
int main( int argc, char *argv[] )
{
  printf( "Hello %s\n", getenv( ”NAME" ) );
  return EXIT_SUCCESS;
}
```

```
$ NAME=”David”
$ ./program
Hello (null)
```

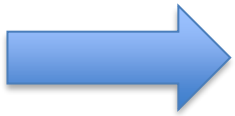
Oops. That’s a mistake.

Exporting Variables

- Variables aren't automatically copied to programs.
  - But, the shell keeps a list of variables to export

Shell

Export	Var	Value
✓	USER	dbsturgi
	PHONE	515-555-0123
✓	NAME	David



Program you start

Var	Value
USER	dbsturgi
NAME	David

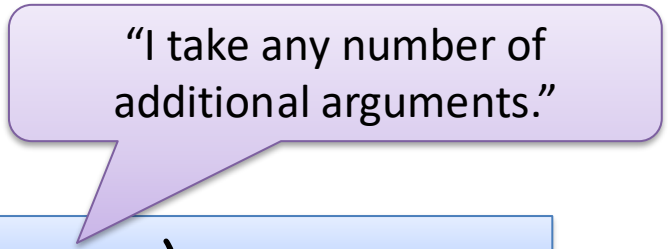
```
$ NAME="David"
$ export NAME
$ ./program
Hello David
```

That's better.

Variable Numbers of Arguments

- We can define functions that take a variable number of arguments
  - printf() needs this
  - The notation looks like this:





- Typically, you’ll have:
  - One or more named arguments (at least one is required)
  - Followed by a variable number of unnamed arguments

Processing Variable Arguments

- We can handle variable numbers of arguments … but we'll need some help.

```
#include <stdarg.h>
```

- We'll need to do four things
  - Create a variable of type `va_list`, to store vararg state
  - Call `va_start()`, to start processing variable arguments
  - Call `va_arg()` for each argument, to extract it with the right type
  - Cll `va_end()` when we're done

Counting Arguments

- There's no automatic mechanism to tell how many arguments we got
- We'll need to build-in a way to do that
  - Like printf(), we could use one of the named arguments to figure it out
  - Or, we could use a mechanism to mark the last argument
- Let's try both of these …

Adding Multiple Arguments

```
int addIntegers( int n, ... )
{
  va_list ap;

  va_start( ap, n );

  int total = 0;
  for ( int i = 0; i < n; i++ )
    total += va_arg( ap, int );

  va_end( ap );

  return total;
}
```

State for vararg processing.

Start processing after n.

Get the next int argument.

All done.

```
int sum = addIntegers( 5, 17, 42, 98, 3, 27 );
```

```
int sum2 = addIntegers( 29, 17, 42, 98, 3, 27 );
```

It's our job to use the function properly.

Printing Multiple Strings

```
void printStrings( char *first, ... )
{
  va_list ap;
  va_start( ap, first );

  printf( "%s\n", first );

  char *str;
  while ( ( str = va_arg( ap, char * ) ) ) {
    printf( "%s\n", str );
  }

  va_end( ap );
}
```

We must have at least one named parameter.

Keep going until we hit the NULL.

```
printStrings( "one", "two", "three", "four", NULL );
```

A null to mark the end.

Max of Multiple Values

```
double maximum( int n, double v1, ... )
{
  va_list ap;
  va_start( ap, v1 );

  double largest = v1;

  for ( int i = 1; i < n; i++ ) {
    double v = va_arg( ap, double );
    if ( v > largest )
      largest = v;
  }

  va_end( ap );
  return largest;
}
```

How many values?

Force the caller to give us at least one.

Handle the first value as a named argument.

Handle the rest through varargs.

```
max = maximum( 6, 32.1, 15.32, 2.99, 45.2, 13.62, 7.0 );
```

Nonlocal Jumps

- A goto will let you jump anywhere
  - … as long as it's in the same function.
  - Maybe an OK mechanism for handling errors
- C also supports a long jump
  - To jump out of a function back to one of its callers
  - Made for handling errors
  - Kind of like an exception in Java

Nonlocal Jumps

- Supported by the setjmp.h header.
- Before you can longjmp, you must take a picture of the place you want to jump back to.
  - This is called your program's environment
  - Where it's executing
  - And where the top of the stack is
- The environment must be stored in instance of jmp_buf

```
jmp_buf env;
```

Taking a Picture

- The setjmp() macro saves the state of the environment.

```
setjmp( env );
```

- Initially, it returns zero.
- You need to check this (for later).

```
if ( setjmp( env ) == 0 ) {
  ..
```

Hitting Rewind

- If something goes wrong, calling longjmp() will take you back to that picture.

```
if ( something_went_wrong )
  longjmp( env, 1 );
```

Some value you can use to indicate what went wrong.

- You can allocate the environment statically
- Or pass it to the function for it to use.
- But, you can only longjmp() **up the stack.**

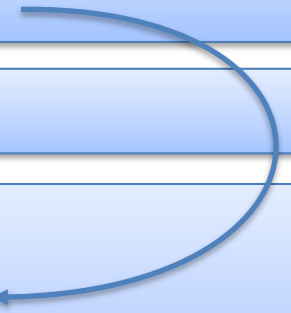
Waking up in setjmp()

- The longjmp() takes you right back to the matching setjmp() call

```
if ( something_went_wrong )
  longjmp( env, 1 );
```

```
Some other function.
```

```
jmp_buf env;
if ( setjmp( env ) == 0 ) {
   Call some other function.
```



- So it looks (almost) like you're right back where you started.

Waking up in setjmp()

- After a longjmp(), the setjmp() won't return zero.
  - It returns the value you passed to longjmp()

```
if ( something_went_wrong )
  longjmp( env, 1 );
```

```
Some other function.
```

```
jmp_buf env;
if ( setjmp( env ) == 0 ) {
   Call some other function.
} else {
   Error handling code.
}
```

I'll return 1 this time.

Using setjmp()/longjmp()

- We can use this kind of like an exception.

```
void middleFunction( jmp_buf *eenv )
{
  int sum = readValues( eenv );
  printf( "Sum: %d\n", sum );
}
```

```
void topFunction()
{
  jmp_buf eenv;

  if ( setjmp( eenv ) == 0 ) {
    middleFunction( &eenv );
  } else {
    printf( "Invalid input!\n" );
  }
}
```

Plan ahead for where to go if something fails.

Passing the error-handling environment among functions.

An error in here could jump past this middle function.

Using setjmp()/longjmp()

- Here's where we notice the error.

```
int readValues( jmp_buf *eenv )
{
  int sum = 0;
  int m, v;

  while ( ( m = scanf( "%d", &v ) ) == 1 )
    sum += v;

  if ( m != EOF )
    longjmp( *eenv, 1 );

  return sum;
}
```

Jump back to the error-handling code.

Skipping the printout in the middle function.

Cleaning Up

- What if we had some resources to free?

```
int readValues( int *list, int capacity, FILE *fp, jmp_buf *eenv )
```

```
void middleFunction( jmp_buf *eenv )
{
  FILE *fp = fopen( "input.txt", "r" );
  if ( !fp )
    longjmp( *eenv, 1 );

  int capacity = 100;
  int *list = (int *)malloc( capacity * sizeof( int ) );

  int vcount = readValues( list, capacity, fp, eenv );

  for ( int i = vcount - 1; i >= 0; i-- )
    printf( "%d\n", list[ i ] );

  fclose( fp );
  free( list );
}
```

Now, this function reads from a file … into an array.

A jump past this function would leak these resources.

We need something like a finally block for this.

Cleaning Up

- We can catch and re-throw to free resources.
- With no changes to the other functions.

```
void middleFunction( jmp_buf *parentEnv )
{
  FILE *fp = fopen( "input.txt", "r" );
  if ( !fp ) longjmp( *parentEnv, 1 );
  int *list = (int *)malloc( 100 * sizeof( int ) );

  jmp_buf midEnv;
  if ( setjmp( midEnv ) == 0 ) {
    int vcount = readValues( list, capacity, fp, &midEnv );
    ... print the list …
    fclose( fp );
    free( list );
  } else {
    fclose( fp );
    free( list );
    longjmp( *parentEnv, 1 );
  }
}
```

Our own environment, to catch the jump.

Tell the readValues() function to go here on an error.

Clean up our part, and jump on up the stack.